

Parallel Computing with Python

Xiaoyuan Suo
Webster University

What is Parallel Computing?

- “If I can shovel my driveway in 1 hour, how long would you expect it to take if someone equally capable were to help me?”
- “If one person can dig a hole with a shovel in 100 seconds, how long will it take for 10 people to dig that hole?”
- “Suppose the goal is to plant 100 trees. Each tree requires that you dig a hole, place the tree in the hole, and refill the hole. Assume each of these steps takes 100 seconds.
 - How long does it take for one person to plant 1 tree?
 - How about 100 trees?
 - Suppose 10 people helped plant trees. How long will it take?”



What is Parallel Computing?

- Parallel computing is a model that divides a task into multiple sub-tasks and executes them simultaneously to increase the speed and efficiency.
 - A problem is broken down into multiple parts. Each part is then broken down into a number of instructions.
 - These parts are allocated to different processors which execute them simultaneously.
 - This increases the speed of execution of programs as a whole.

What is Distributed Computing?

- Distributed computing is a field that studies distributed systems.
- Distributed systems are systems that have multiple computers located in different locations.
- These computers in a distributed system work on the same program.
- The program is divided into different tasks and allocated to different computers.
- The computers communicate with the help of message passing. Upon completion of computing, the result is collated and presented to the user.

Difference between parallel and distributed computing?

- Parallel computing generally requires one computer with multiple processors.
 - Multiple processors within the same computer system execute instructions simultaneously.
 - All the processors work towards completing the same task. Thus they have to share resources and data.
- In distributed computing, several computer systems are involved.
- In parallel computing, the tasks to be solved are divided into multiple smaller parts. These smaller tasks are assigned to multiple processors.
- In parallel computing environments, the number of processors you can add is restricted. Distributed computing environments are more scalable.
- In systems implementing parallel computing, all the processors share the same memory. Distributed systems, on the other hand, have their own memory and processors.
- Parallel computing is often used in places requiring higher and faster processing power. For example, supercomputers.



Why PDC?

- The primary goal of parallel computing is to increase available computation power for faster application processing and problem solving.
 - It will save you time
 - It will allow you to solve larger problems.
 - It's here whether you like it or not!
- Parallel and distributed computing (PDC) can be found in research and industries of many science and engineering domains.
 - You will have a good chance to obtain better internships if you master the concepts well



Thread Basics

- Define a Thread
 - Sample program demo in class

```
import threading #

def function(i):
    print (f'function called by thread {i}')

threads = []

for i in range(5):
    t = threading.Thread(target=function, args = (i,))
    threads.append(t)
    t.start()
    t.join()
```

How to determine the current thread

```
.py x builtins.py x phonebook.py x create_phonebook_db.py x __init__.py
import threading
import time
def first_function():
    print(threading.current_thread().getName() + \
          str(' is Starting '))
    time.sleep(2)
    print(threading.current_thread().getName() + \
          str(' is Existing '))

def second_function():
    print(threading.current_thread().getName() + \
          str(' is Starting '))
    time.sleep(2)
    print(threading.current_thread().getName() + \
          str(' is Existing '))

def second_function():
    print(threading.current_thread().getName() + \
          str(' is Starting '))
    time.sleep(2)
    print(threading.current_thread().getName() + \
          str(' is Existing '))

if __name__ == '__main__':
    t1 = threading.Thread(name='first_function', target=first_function)
    t2 = threading.Thread(name='second_function', target=first_function)
    t3 = threading.Thread(name='third_function', target=first_function)

    t1.start()
    t2.start()
    t3.start()

    t1.join()
    t2.join()
    t3.join()
```

Use Thread in SubClass

```
import threading
import time

exitFlag = 0

class myThread(threading.Thread):
    def __init__(self, threadID, name, counter):
        threading.Thread.__init__(self)
        self.threadID = threadID
        self.name = name
        self.counter = counter

    def run(self):
        print('Starting ' + self.name)
        print_time(self.name, self.counter, 5)
        print('Existing ' + self.name)

def print_time(threadName, delay, counter):
    while counter:
        #if exitFlag:
        #    threading.exit()
        time.sleep(delay)
        print(f'{threadName}: {time.ctime(time.time())}')
        counter -= 1

thread1 = myThread(1, "Thread-1", 1)
thread2 = myThread(2, "Thread-2", 2)

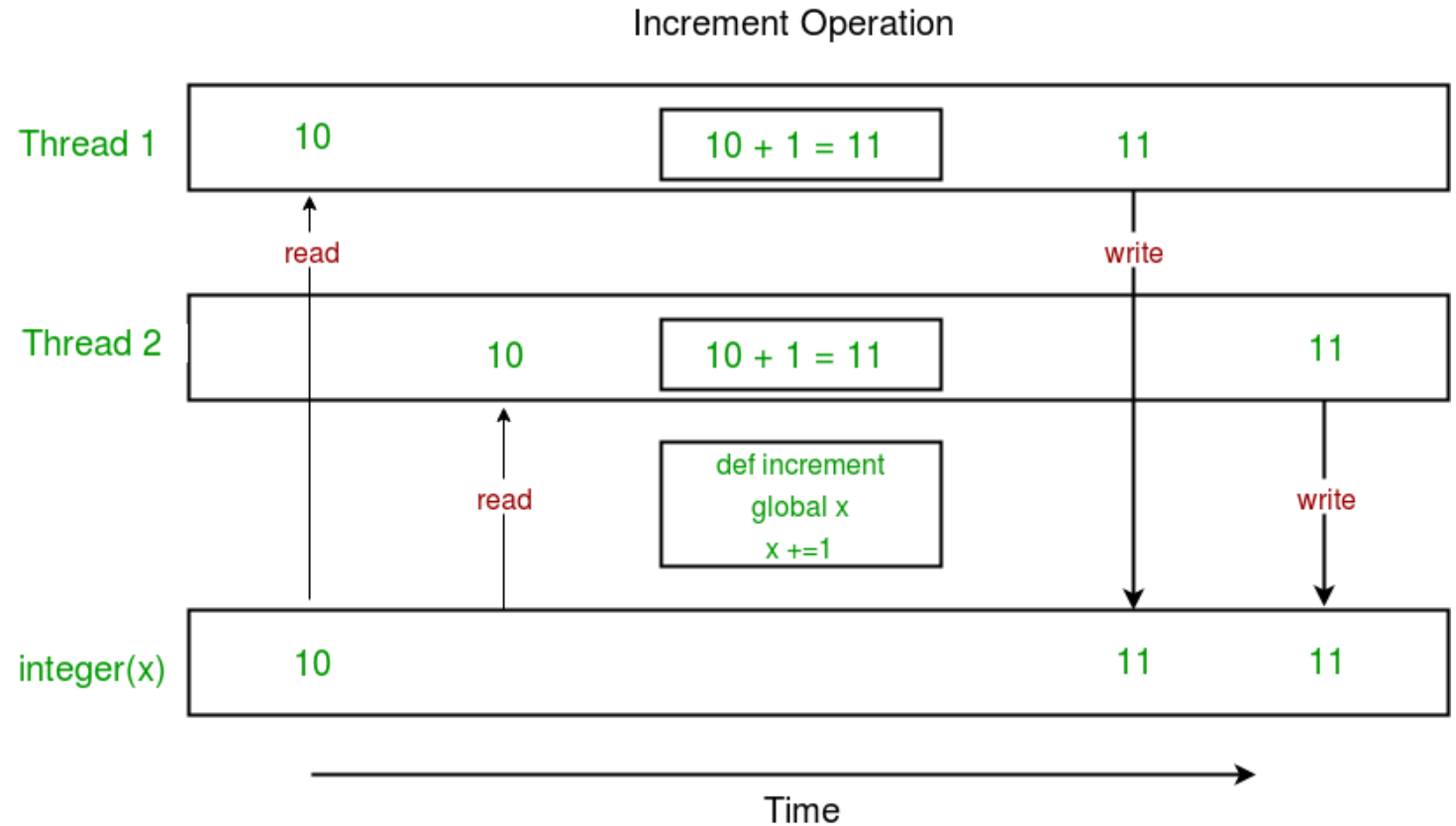
thread1.start()
thread2.start()

thread1.join()
thread2.join()

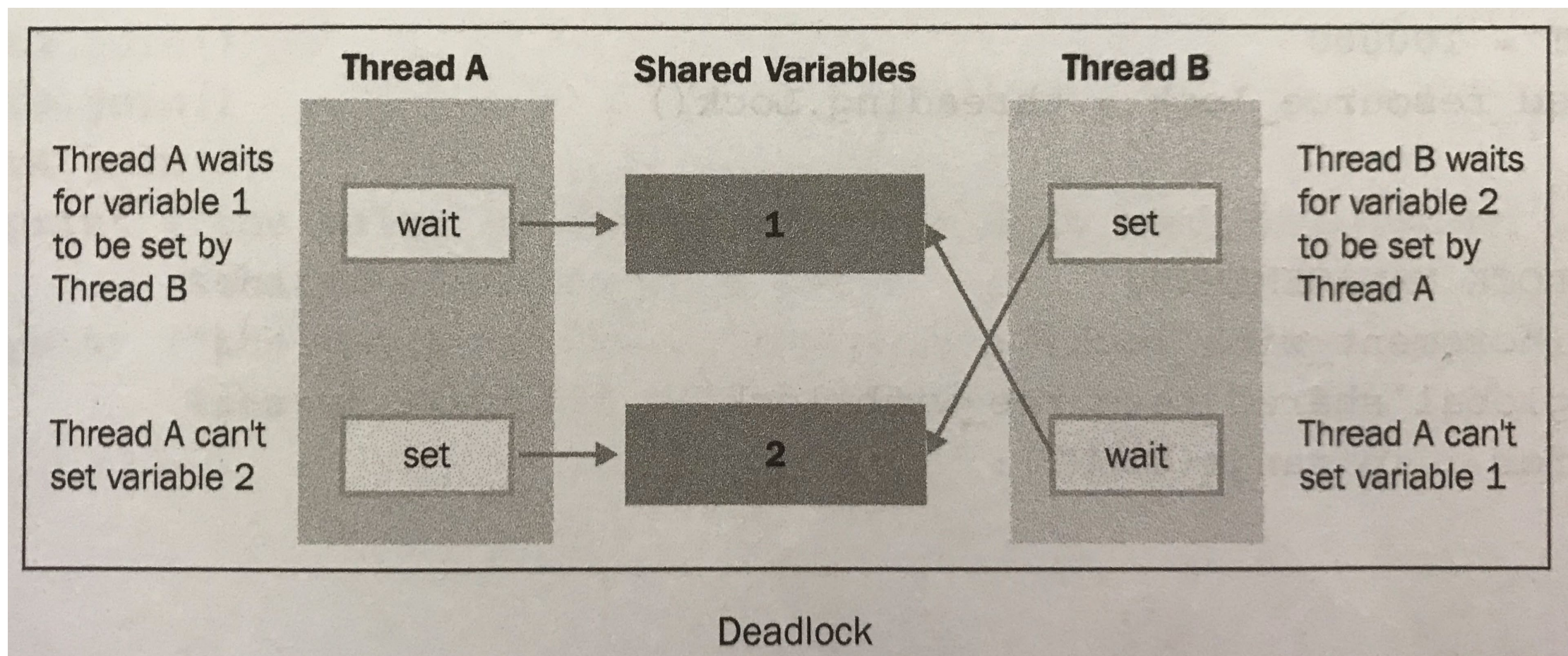
print('existing..')
```

Race Condition

- Concurrent accesses to shared resource can lead to **race condition**.
 - A race condition occurs when two or more threads can access shared data and they try to change it at the same time. As a result, the values of variables may be unpredictable and vary depending on the timings of context switches of the processes
- Run a sample Program
 - Race_condition5.py



Thread Synchronization with Lock and RLock



Lock management - Data Dependency

```
import threading

shared_resource_with_lock = 0
shared_resource_without_lock = 0

count = 100000

share_resource_lock = threading.Lock()

# lock management

def increment_with_lock():
    global shared_resource_with_lock #modifying global variable
    for i in range(count):
        share_resource_lock.acquire()
        shared_resource_with_lock += 1
        share_resource_lock.release()

def decrement_with_lock():
    global shared_resource_with_lock
    for i in range(count):
        share_resource_lock.acquire()
        shared_resource_with_lock -= 1
        share_resource_lock.release()
```

```
#no lock management

def increment_without_lock():
    global shared_resource_without_lock
    for i in range(count):
        shared_resource_without_lock += 1

def decrement_without_lock():
    global shared_resource_without_lock
    for i in range(count):
        shared_resource_without_lock -= 1
```

Lock Management cont.

```
if __name__ == '__main__':  
    t1 = threading.Thread(target=incrememnt_with_lock)  
    t2 = threading.Thread(target=decrement_with_lock)  
    t3 = threading.Thread(target=increment_without_lock)  
    t4 = threading.Thread(target=decrement_without_lock)  
  
    t1.start()  
    t2.start()  
    t3.start()  
    t4.start()  
  
    t1.join()  
    t2.join()  
    t3.join()  
    t4.join()  
  
    print("The value of shared variable with lock management is: ", shared_resource_with_lock)  
    print("The value of shared variable without lock management is: ",  
          shared_resource_without_lock)
```

/Library/Frameworks/Python.framework/Versions/3.8/bin/python3.8 /Users/user/PycharmProjects/untitled1/test.py

The value of shared variable with lock management is: 0

The value of shared variable without lock management is: -18394

Process finished with exit code 0

Lock management

- In the `increment_with_lock()` and `decrement_with_lock()` functions, you can see how to use lock management. When you need to access the resource, call `acquire()` to hold the lock (this will wait for the lock to be released, if necessary), and call `release()` to release it.

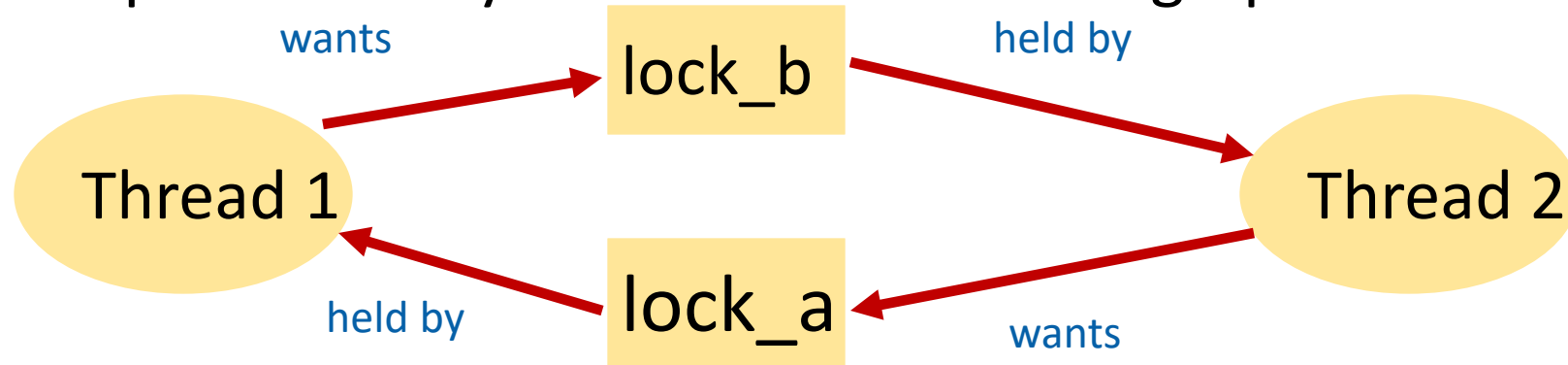
```
share_resource_lock.acquire()  
shared_resource_with_lock -= 1  
share_resource_lock.release()
```

How it works

- Locks have two states: locked and unlocked
- We have two methods that are used to manipulate the locks: `acquire()` and `release()`
- The following are the rules:
 - If the state is unlocked, a call to `acquire()` changes the state to locked.
 - If the state is locked, a call to `acquire()` blocks until another thread calls `release()`
 - If the state is unlocked, a call to `release()` raises a `RuntimeError` exception
 - If the state is locked, a call to `release()` changes the state to unlocked.

Problem--Deadlock

- A situation involving two or more threads (processes) in which no thread may proceed because each is waiting for a resource held by another
- Can be represented by a resource allocation graph





Four Conditions for Deadlock

- Mutually exclusive access to a resource
 - Mutual exclusion is a property of concurrency control, which prevents race conditions.
 - It is the requirement that one thread of execution never enters its critical section at the same time that another concurrent thread of execution enters its own critical section, which refers to an interval of time during which a thread of execution accesses a shared resource, such as shared memory.
- Threads hold onto resources they have while they wait for additional resources
- Resources cannot be taken away from threads
- Cycle in resource allocation graph



Python Rlock

- If we want only the thread that acquires a lock to release it, we must use Rlock objects
- Rlock() is useful when you want to have a thread-safe access from outside the class and use the same methods from inside the class

Python RLock

```
import threading
import time

class Box(object):
    lock = threading.RLock()

    def __init__(self):
        self.total_items = 0

    def execute(self, n):
        Box.lock.acquire()
        self.total_items += n
        Box.lock.release()

    def add(self):
        Box.lock.acquire()
        self.execute(1)
        Box.lock.release()

    def remove(self):
        Box.lock.acquire()
        self.execute(-1)
        Box.lock.release()
```

#these two functions run n in separate threads and call the Box's methods

```
def adder(box, items):
    while items > 0:
        print('adding 1 item in the box')
        box.add()
        time.sleep(2)
        items -= 1

def remover(box, items):
    while items > 0:
        print('removing 1 item in the box')
        box.remove()
        time.sleep(2)
        items -= 1
```

#the main program bulid some threads to validify

```
if __name__ == '__main__':
    item = 5
    print(f'putting {item} items in the box')
    box = Box()
    t1 = threading.Thread(target=adder, args=(box, item))
    t2 = threading.Thread(target=remover, args=(box, item))

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    print(f'{box.total_items} still remain in the box ')
```

Python Rlock cont.

- Two threads t1, t2 are with the associated functions adder() and remover()
 - The functions are active when the number of items is greater than 0
- The two functions adder() and remover() interact with the items of the Box class, respectively, and call the Box class methods: add() and remove().
 - In each function call, a resource is captured and then released.

Python Multiprocessing

```
import multiprocessing

def func (i):
    print(f'called func in process {i}')

if __name__ == '__main__':
    processed_jobs = []
    for i in range (5):
        p = multiprocessing.Process(target=func, args=(i,))
        processed_jobs.append(p)
        p.start()
        p.join()
```

Multiprocessing

```
# importing the multiprocessing module
import multiprocessing

def print_cube(num):
    """
    function to print cube of given num
    """
    print("Cube: {}".format(num * num * num))

def print_square(num):
    """
    function to print square of given num
    """
    print("Square: {}".format(num * num))

if __name__ == "__main__":
    # creating processes
    p1 = multiprocessing.Process(target=print_square, args=(10, ))
    p2 = multiprocessing.Process(target=print_cube, args=(10, ))

    # starting process 1
    p1.start()
    # starting process 2
    p2.start()

    # wait until process 1 is finished
    p1.join()
    # wait until process 2 is finished
    p2.join()

    # both processes finished
    print("Done!")
```

Multiprocessing with a class

```
import multiprocessing
import time

class Process(multiprocessing.Process):
    def __init__(self, id):
        super(Process, self).__init__()
        self.id = id

    def run(self):
        time.sleep(1)
        print("I'm the process with id: {}".format(self.id))

if __name__ == '__main__':
    p = Process(0)
    p.start()
    p.join()
    p = Process(1)
    p.start()
    p.join()
```

Multiprocessing with the Pool class

- Pool class can be used for parallel execution of a function for different input data. The multiprocessing.Pool() class spawns a set of processes called workers and can submit tasks using the methods apply/apply_async and map/map_async. For parallel mapping, you should first initialize a multiprocessing.Pool() object. The first argument is the number of workers; if not given, that number will be equal to the number of cores in the system.

```
pool = multiprocessing.Pool()  
pool = multiprocessing.Pool(processes=4)
```

Synchronous and Asynchronous Execution

- A synchronous execution is one the processes are completed in the same order in which it was started. This is achieved by locking the main program until the respective processes are finished.
- Asynchronous, on the other hand, doesn't involve locking. As a result, the order of results can get mixed up but usually gets done quicker.
- Pool Class
 - Synchronous execution
 - `Pool.map()` and `Pool.starmap()`
 - `Pool.apply()`
 - Asynchronous execution
 - `Pool.map_async()` and `Pool.starmap_async()`
 - `Pool.apply_async()`

Pool class

- Let see by an example. In this example, we will see how to pass a function which computes the square of a number. Using Pool.map() you can map the function to the list and passing the function and the list of inputs as arguments, as follows:

```
def square(x):  
    return x * x  
  
inputs = [0, 1, 2, 3, 4]  
outputs = pool.map(square, inputs)
```

```
import multiprocessing  
import time  
  
def square(x):  
    return x * x  
  
if __name__ == '__main__':  
    pool = multiprocessing.Pool()  
    pool = multiprocessing.Pool(processes=4)  
    inputs = [0,1,2,3,4]  
    outputs = pool.map(square, inputs)  
    print("Input: {}".format(inputs))  
    print("Output: {}".format(outputs))
```

Asynchronous output

- When we use the normal map method, the execution of the program is stopped until all the workers completed the task.
Using `map_async()`, the `AsyncResult` object is returned immediately without stopping the main program and the task is done in the background. The result can be retrieved by using the `AsyncResult.get()` method at any time as shown below:

```
import multiprocessing
import time

def square(x):
    return x * x

if __name__ == '__main__':
    pool = multiprocessing.Pool()
    inputs = [0,1,2,3,4]
    outputs_async = pool.map_async(square, inputs)
    outputs = outputs_async.get()
    print("Output: {}".format(outputs))
```

```
outputs_async = pool.map_async(square, inputs)
outputs = outputs_async.get()
```

Example

A large set of data

```
# Prepare data
np.random.RandomState(100)
arr = np.random.randint(0, 10, size=[200000, 5])
data = arr.tolist()
print(data[:5])
```

```
#without parallelization
def findmin(row):
    min = row[0]
    for n in row:
        if min > n:
            min = n
    return min

if __name__ == '__main__':
    pool = mp.Pool(mp.cpu_count())

    results = pool.map(findmin, [row for row in data])

    pool.close()

    final_result = findmin(results)
    print(final_result)
```

Assignment 12

- Implement a parallel version of “find_sum”,
 - Given a large set of randomly generated numbers (5 rows and 10,000,000 columns per row)
 - Find the sum

Useful resources:

- <https://www.hitechnectar.com/blogs/distributed-vs-parallel-computing/>
- Python parallel programming cookbook by Giancarlo Zaccone
- The OpenMP common core
- Distributed computing with Python